

JAVA SWING BÁSICO

MG DIEGO ALONSO IQUIRA BECERRA

EXPOSICIÓN

Se realizara la exposición de la guía de laboratorio realizadas la clase anterior.



GUI VS CUI

¿Que es la GUI y que es el CUI?



CUI

El CUI (Interfaz de Usuario de Línea de Comandos, por sus siglas en inglés Command-Line User Interface) es un tipo de interfaz en la que los usuarios interactúan con un sistema informático o una aplicación a través de la escritura de comandos de texto en una terminal o consola

El usuario debe ingresar comandos, a menudo seguidos de parámetros o argumentos, para interactuar con el sistema. No hay elementos visuales como botones o ventanas.

GUI

La GUI (Interfaz Gráfica de Usuario) es una forma de interacción entre los usuarios y un sistema informático o una aplicación mediante elementos visuales, como ventanas, botones, iconos, menús, y otros componentes gráficos.

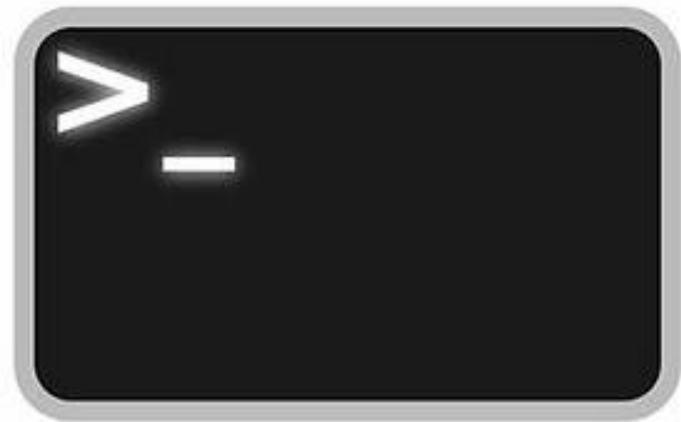
Es decir, la GUI permite que el usuario interactúe con un programa sin necesidad de escribir comandos en una línea de texto, lo que hace que las aplicaciones sean más fáciles de usar y accesibles para una mayor cantidad de personas.

GUI

GUI



CUI



ENTRADA/SALIDA

La entrada/salida simple basada en GUI se refiere a cómo un programa puede interactuar con el usuario a través de una Interfaz Gráfica de Usuario (GUI) para recibir datos de entrada y mostrar resultados o salidas visualmente, utilizando componentes gráficos como campos de texto, botones, etiquetas, etc.

ENTRADA/SALIDA

La mayoría de las aplicaciones que usamos a diario utilizan ventanas o cuadros de diálogo (también conocidos como diálogos) para interactuar con el usuario.

Los cuadros de diálogo son ventanas en las cuales los programas muestran mensajes importantes al usuario, u obtienen información de éste.

La clase JOptionPane de Java (paquete javax.swing) proporciona cuadros de diálogo prefabricados para entrada y salida.

JOPTIONPANE

```
import javax.swing.JOptionPane;
public class App {
    public static void main(String[] args) throws Exception {
        // obtiene la entrada del usuario de los diálogos de entrada de
        JOptionPane
        String primerNumero =
        JOptionPane.showInputDialog("Introduzca el primer entero");
        String segundoNumero =
        JOptionPane.showInputDialog("Introduzca el segundo entero");
        // convierte las entradas String en valores int para usarlos en un
        cálculo
        int numero1 = Integer.parseInt(primerNumero);
        int numero2 = Integer.parseInt(segundoNumero);
        int suma = numero1 + numero2;
        // muestra los resultados en un diálogo de mensajes de JOptionPane
        JOptionPane.showMessageDialog(null, "La suma es " + suma, "Suma de dos
        enteros",
        JOptionPane.PLAIN_MESSAGE);
    }
}
```

JFRAME

JFrame es una de las clases más fundamentales en la creación de interfaces gráficas en Java utilizando Swing.

Representa una ventana o marco en la que puedes agregar componentes gráficos como botones, campos de texto, etiquetas, etc.

Es la "estructura" básica sobre la que se construye toda la interfaz gráfica.

CARACTERÍSTICAS DE JFRAME

- **Contenedor Principal:** JFrame es el contenedor principal de la ventana.
 - Al crear una instancia de JFrame, estamos creando una ventana sobre la que podemos añadir otros componentes.
- **Cierre de la Ventana:** Se puede configurar para que la aplicación se cierre cuando el usuario cierre la ventana.
 - Esto se logra con el método
 - `setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)`.
- **Tamaño:** Puedes definir el tamaño de la ventana con `setSize(width, height)` o `setPreferredSize(Dimension)`.
- **Visibilidad:** Para que la ventana sea visible, se debe llamar a `setVisible(true)`.

CODIGO

```
import javax.swing.JFrame;

public class GUIJFrame {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Mi Ventana");
        frame.setSize(400, 300); // Establecer el tamaño de la
ventana
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); //
Configura el comportamiento al cerrar la ventana
        frame.setVisible(true); // Hacer que la ventana sea visible
    }
}
```

JPANEL

JPanel es un contenedor que se usa para organizar y agrupar otros componentes gráficos dentro de una ventana (en este caso, dentro de un JFrame).

Es uno de los componentes más útiles para estructurar una interfaz gráfica, ya que te permite organizar otros componentes como botones, campos de texto, etiquetas, etc.

CARACTERÍSTICAS DE JPANEL

- **Organización de Componentes:** JPanel se utiliza para agrupar y organizar otros componentes dentro de la interfaz.
 - Por ejemplo, puedes tener varios paneles para distintas secciones de tu aplicación.
- **Layout Managers:** Puedes usar diferentes Layout Managers con el JPanel para organizar los componentes dentro de él (como FlowLayout, BorderLayout, GridLayout, etc.).
- **Reutilización:** Puedes crear paneles personalizados y agregarlos a otras interfaces gráficas o ventanas, lo que permite que tu interfaz sea modular y más fácil de manejar.

CODIGO

```
import java.awt.FlowLayout;
```

```
import javax.swing.JFrame;  
import javax.swing.JPanel;
```

```
public class GUIJFrame {  
    public static void main(String[] args) {  
        JFrame frame = new JFrame("Mi Ventana");  
        frame.setSize(400, 300); // Establecer el tamaño de la ventana  
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // Configura el  
        comportamiento al cerrar la ventana  
        frame.setVisible(true); // Hacer que la ventana sea visible  
        JPanel panel = new JPanel(); // Crear un panel  
        panel.setLayout(new FlowLayout()); // Establecer el Layout Manager  
        frame.add(panel); // Añadir el panel al JFrame  
    }  
}
```

COMPONENTES DE ENTRADA Y SALIDA

Los componentes de entrada y salida son los elementos de la interfaz gráfica con los que el usuario interactúa para ingresar datos (entrada) y mostrar resultados o mensajes (salida).

JTEXTFIELD

El JTextField (Campo de texto de una línea) se utiliza para que el usuario ingrese texto en una sola línea.

Es comúnmente utilizado para entradas como nombres, direcciones o cualquier tipo de información corta.

```
JTextField textField = new JTextField(20);
```

JPASSWORDFIELD

El JPasswordField (Campo de texto para contraseñas), es similar a JTextField, pero diseñado para entradas de contraseñas, en donde los caracteres se enmascaran (se muestran como puntos o asteriscos).

```
JPasswordField passwordField = new  
JPasswordField(20);
```

JTEXTAREA

El JTextArea (Área de texto multi-línea) se utiliza para que el usuario ingrese texto de varias líneas, como comentarios o descripciones largas.

```
JTextArea textArea = new JTextArea(5, 20); // 5 filas, 20  
columnas
```

JCOMBOBOX

El JComboBox (Caja combinada o menú desplegable) permite al usuario seleccionar una opción de una lista predefinida de elementos.

Es útil para limitar las opciones de entrada a un conjunto determinado de valores.

```
String[] opciones = {"Opción 1", "Opción 2", "Opción 3"};  
JComboBox<String> comboBox = new  
JComboBox<>(opciones);
```

JLABEL

El JLabel (Etiqueta de texto) se utiliza para mostrar texto o imágenes en la interfaz.

Puede servir para mostrar mensajes de salida, como resultados, instrucciones o etiquetas.

Las etiquetas no permiten edición por parte del usuario.

```
JLabel label = new JLabel("Bienvenido a mi aplicación!");
```

JTEXTAREA

El JTextArea (Área de texto multi-línea para salida) es similar al componente de entrada JTextArea, pero se usa para mostrar múltiples líneas de texto como resultado de una operación.

```
JTextArea salidaArea = new JTextArea(5, 20); // 5 filas, 20  
columnas
```

```
salidaArea.setText("Resultado del proceso");
```

JOPTIONPANE

El JOptionPane (Ventana de mensaje emergente) se usa para mostrar cuadros de diálogo con mensajes informativos o de error.

```
JOptionPane.showMessageDialog(frame, "Este es  
un mensaje de salida", "Título",  
JOptionPane.INFORMATION_MESSAGE);
```

COMPONENTES DE ENTRADA Y SALIDA

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class EntradaSalidaGUI {
    public static void main(String[] args) {
        // Crear la ventana
        JFrame frame = new JFrame("Entrada/Salida Simple");
        frame.setSize(400, 200); // Tamaño de la ventana
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        // Crear un panel para organizar los componentes
        JPanel panel = new JPanel();
        frame.add(panel);
        panel.setLayout(new FlowLayout());

        // Crear los componentes
        JLabel etiqueta = new JLabel("Introduce tu nombre: ");
        JTextField campoTexto = new JTextField(20); // Campo de texto para ingresar el nombre
        JButton botonSaludar = new JButton("Saludar"); // Botón para generar la salida
        JLabel salida = new JLabel(""); // Etiqueta para mostrar el mensaje de salida

        // Agregar los componentes al panel
        panel.add(etiqueta);
        panel.add(campoTexto);
        panel.add(botonSaludar);
        panel.add(salida);

        // Acción cuando se hace clic en el botón
        botonSaludar.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                String nombre = campoTexto.getText(); // Obtener el texto ingresado
                if (!nombre.trim().isEmpty()) {
                    salida.setText("¡Hola, " + nombre + "!"); // Mostrar mensaje de salida
                } else {
                    salida.setText("Por favor, ingresa tu nombre."); // Si el campo está vacío
                }
            }
        });

        // Hacer visible la ventana
        frame.setVisible(true);
    }
}
```


AWT

En realidad, hay dos conjuntos de componentes de GUI en Java. En los primeros días de Java, las GUI se creaban a partir de componentes del Abstract Window Toolkit (AWT) en el paquete `java.awt`.

Éstos se ven como los componentes de GUI nativos de la plataforma en la que se ejecuta un programa de Java.

Por ejemplo, un objeto de tipo `Button` que se muestra en un programa de Java ejecutándose en Microsoft Windows tendrá la misma apariencia que los botones en las demás aplicaciones Windows.

En el sistema operativo Apple Mac OS X, el objeto `Button` tendrá la misma apariencia visual que los botones en las demás aplicaciones Mac.

ACTIVIDAD

De manera grupal escojan una aplicación para la cual van a diseñar un mockup de la interfaz

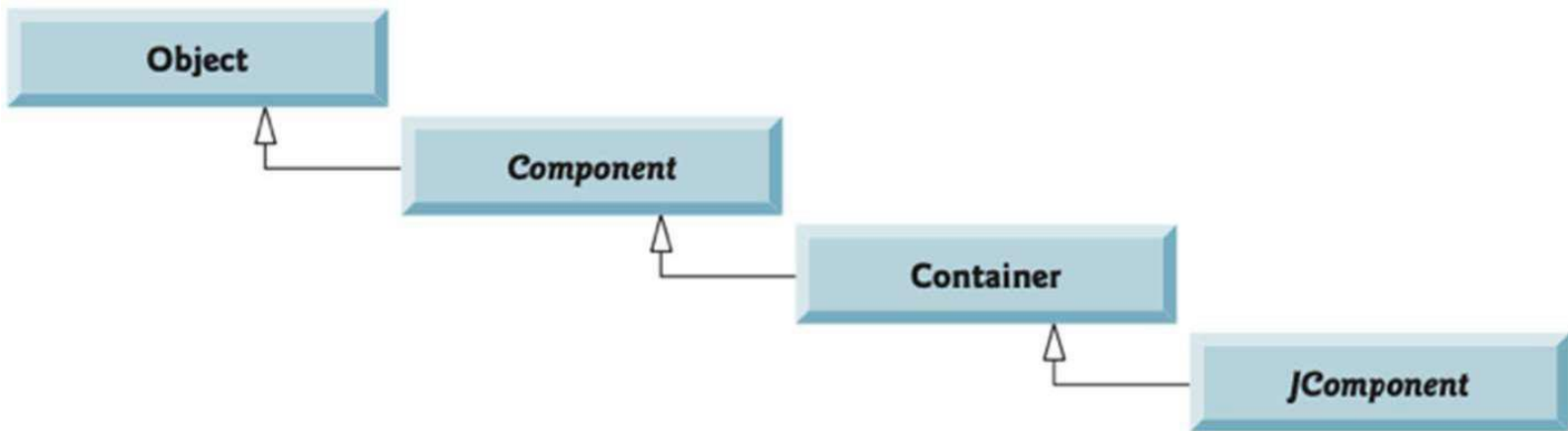
- Una aplicación de gestión de tareas.
- Una aplicación de compras en línea.
- Una aplicación de mensajería.
- Una aplicación de calendario o agenda.
- Una aplicación de noticias.

En una hoja de papel dibujen un rectángulo que simule el contorno de la pantalla, dividan las pantallas en diferentes secciones y distribuyan los elementos.

Creen al menos dos ventanas.

Muestren cómo se interrelacionan las pantallas: Si un usuario hace clic en un botón, ¿qué pasa? Por ejemplo, si presionan un botón de "Agregar tarea", la aplicación puede mostrar una nueva pantalla para ingresar los detalles de la tarea.

JERARQUÍA DE CLASES DE LA CLASE COMPONENTES GUI



GUI

Al crear una GUI, cada componente de ésta debe adjuntarse a un contenedor, como una ventana creada con un objeto JFrame.

Además, por lo general debemos decidir en dónde colocar cada componente de la GUI y establecer su tamaño; esto se conoce como Especificar el Esquema.

Java cuenta con varios administradores de esquemas que pueden ayudarle a colocar los componentes.

Así mismo, hay varios IDEs que permiten realizar el mismo trabajo de un administrador de esquema sólo utilizando el ratón y de manera visual.

APP

```
import javax.swing.JFrame;
public class App
{
    public static void main(String[] args)
    {
        LabelFrame marcoEtiqueta = new LabelFrame();
        marcoEtiqueta.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    };
    marcoEtiqueta.setSize(300, 250);
    marcoEtiqueta.setVisible(true);
}
} // Fin de App Prueba Label
```

LABELFRAME

```
import java.awt.FlowLayout; // especifica cómo se van a ordenar los componentes
import javax.swing.JFrame; // proporciona las características básicas de una ventana
import javax.swing.JLabel; // muestra texto e imágenes
import javax.swing.SwingConstants; // constantes comunes utilizadas con Swing
import javax.swing.ImageIcon; // carga las imágenes
public class LabelFrame extends JFrame
{
    private JLabel etiqueta1; // JLabel sólo con texto
    private JLabel etiqueta2; // JLabel construida con texto y un icono
    private JLabel etiqueta3; // JLabel con texto adicional e icono 15
    // El constructor de LabelFrame agrega objetos JLabel a JFrame
    public LabelFrame()
    {
        // Constructor de JLabel con un argumento String
        super("Prueba de JLabel");
        setLayout(new FlowLayout()); // establece el esquema del marco
        // Constructor de JLabel con un argumento String
        etiqueta1 = new JLabel("Etiqueta con texto");
        etiqueta1.setToolTipText("Esta es etiqueta1");
        add(etiqueta1); // agrega etiqueta1 a JFrame
        // Constructor de JLabel con argumentos de cadena, Icono y alineación
        ImageIcon insecto = new ImageIcon("C:\\imagen\\insecto.png");
        etiqueta2 = new JLabel("Etiqueta con texto e icono", insecto,
            SwingConstants.LEFT);
        etiqueta2.setToolTipText("Esta es etiqueta2");
        add(etiqueta2); // agrega etiqueta2 a JFrame
        etiqueta3 = new JLabel(); // constructor de JLabel sin argumentos
        etiqueta3.setText("Etiqueta con icono y texto en la parte inferior");
        etiqueta3.setIcon(insecto); // agrega icono a JLabel
        etiqueta3.setHorizontalTextPosition(SwingConstants.CENTER);
        etiqueta3.setVerticalTextPosition(SwingConstants.BOTTOM);
        etiqueta3.setToolTipText("Esta es etiqueta3");
        add(etiqueta3); // agrega etiqueta3 a JFrame
    }
} // fin de la clase LabelFrame
```

MANEJO DE EVENTOS

Las GUI son controladas por eventos. Cuando el usuario interactúa con un componente de la GUI, la interacción (conocida como un evento) controla el programa para que realice una tarea.

Algunas interacciones o eventos comunes del usuario que podrían hacer que una aplicación realizara una tarea incluyen:

- El hacer clic en un botón
- Escribir en un campo de texto
- Seleccionar un elemento de un menú
- Cerrar una ventana
- Mover el ratón.

APP

```
import javax.swing.JFrame;
public class App
{
    public static void main(String[] args)
    {
        CampoTextoMarco campoTextoMarco = new CampoTextoMarco();
        campoTextoMarco.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        campoTextoMarco.setSize(300, 250);
        campoTextoMarco.setVisible(true);
    }
} // Fin de App Prueba Label
```


CAMPOTEXTOMARCO

```
package prueba;
```

```
//Archivo "CampoTextoMarco.java"
//Los componentes JTextField y JPasswordField.
import java.awt.FlowLayout;
import java.awt.event.ActionListener;
import java.awt.event.ActionEvent;
import javax.swing.JFrame;
import javax.swing.JTextField;
import javax.swing.JPasswordField;
import javax.swing.JOptionPane;
public class CampoTextoMarco extends JFrame {
    private final JTextField campoTexto1; // campo de texto con tamaño fijo
    private final JTextField campoTexto2; // campo de texto con texto
    private final JTextField campoTexto3; // campo de texto con texto y tamaño
    private final JPasswordField campoContrasenia; // campo de contraseña con
    texto
    //El constructor de CampoTextoMarco agrega objetos JTextField a JFrame
    public CampoTextoMarco()
    {
        super("Prueba de JTextField y JPasswordField");
        setLayout(new FlowLayout());
        //construye campo de texto con 10 columnas
        campoTexto1 = new JTextField(10);
        add(campoTexto1); // agrega campoTexto1 a JFrame
        //construye campo de texto con texto predeterminado
        campoTexto2 = new JTextField("Escriba el texto aqui");
        add(campoTexto2); // agrega campoTexto2 a JFrame
        //construye campo de texto con texto predeterminado y 21 columnas
        campoTexto3 = new JTextField("Campo de texto no editable", 21);
        campoTexto3.setEditable(false); // deshabilita la edición
        add(campoTexto3); // agrega campoTexto3 a JFrame
        //construye campo de contraseña con texto predeterminado
        campoContrasenia = new JPasswordField("Texto oculto");
        add(campoContrasenia); // agrega campoContrasenia a JFrame
        //registra los manejadores de eventos
        ManejadorCampoTexto manejador = new ManejadorCampoTexto();
        campoTexto1.addActionListener(manejador);
        campoTexto2.addActionListener(manejador);
        campoTexto3.addActionListener(manejador);
        campoContrasenia.addActionListener(manejador);
    }
    //clase interna privada para el manejo de eventos
    private class ManejadorCampoTexto implements ActionListener
    {
        //procesa los eventos de campo de texto
        @Override
        public void actionPerformed(ActionEvent evento)
        {

```

```
String cadena = "";
// el usuario oprimió Intro en el objeto JTextField campoTexto1
if (evento.getSource() == campoTexto1)
    cadena = String.format("campoTexto1: %s", evento.getActionCommand());
// el usuario oprimió Intro en el objeto JTextField campoTexto2
else if (evento.getSource() == campoTexto2)
    cadena = String.format("campoTexto2: %s", evento.getActionCommand() );
// el usuario oprimió Intro en el objeto JTextField campoTexto3
else if (evento.getSource() == campoTexto3)
    cadena = String.format("campoTexto3: %s", evento.getActionCommand());
// el usuario oprimió Intro en el objeto JTextField campoContrasenia
else if (evento.getSource() == campoContrasenia)
    cadena = String.format("campoContrasenia: %s", evento.getActionCommand());
// muestra el contenido del objeto JTextField
JOptionPane.showMessageDialog(null, cadena);
} //fin de la clase privada interna ManejadorCampoTexto
} //fin de la clase CampoTextoMarco
```

MANEJO DE EVENTOS DE RATÓN

El paquete `javax.swing.event` contiene la interfaz `MouseListener`, la cual extiende a las interfaces `MouseListener` y `MouseMotionListener` para crear una sola interfaz que contiene todos los métodos de `MouseListener` y `MouseMotionListener`.

Estos métodos se llaman cuando el ratón interactúa con un objeto `Component`, si se registran objetos componentes de escucha de eventos apropiados para ese objeto `Component`.

MÉTODOS MOUSE

Métodos de las interfaces `MouseListener` y `MouseMotionListener`

Métodos de la interfaz `MouseListener`

```
public void mousePressed(MouseEvent evento)
```

Es llamado cuando se *oprime* un botón del ratón, mientras el cursor del ratón está sobre un componente.

```
public void mouseClicked(MouseEvent evento)
```

Es llamado cuando se *oprime y suelta* un botón del ratón, mientras el cursor del ratón permanece estacionario sobre un componente. Este evento siempre va precedido por una llamada a `mousePressed` y `mouseReleased`.

```
public void mouseReleased(MouseEvent evento)
```

Es llamado cuando se *suelta un botón de ratón después de ser oprimido*. Este evento siempre va precedido por una llamada a `mousePressed` y por una o más llamadas a `mouseDragged`.

```
public void mouseEntered(MouseEvent evento)
```

Es llamado cuando el cursor del ratón *entra* a los límites de un componente.

```
public void mouseExited(MouseEvent evento)
```

Es llama cuando el cursor del ratón *sale* de los límites de un componente.

MÉTODOS MOUSE

Métodos de la interfaz MouseMotionListener

```
public void mouseDragged(MouseEvent evento)
```

Es llamado cuando el botón del ratón se *oprime* mientras el cursor del ratón se encuentra sobre un componente y el ratón se *mueve* mientras el botón *sigue oprimido*. Este evento siempre va precedido por una llamada a `mousePressed`. Todos los eventos de arrastre del ratón se envían al componente en el cual empezó la acción de arrastre.

```
public void mouseMoved(MouseEvent evento)
```

Es llamado al *moverse* el ratón (sin oprimir los botones del ratón) cuando su cursor se encuentra sobre un componente. Todos los eventos de movimiento se envían al componente sobre el cual se encuentra el ratón posicionado en ese momento.

MOUSE WHEEL LISTENER

Java también cuenta con la interfaz `MouseWheelListener` para permitir a las aplicaciones responder a la rotación de la rueda de un ratón.

Esta interfaz declara el método `mouseWheelMoved`, el cual recibe un evento `MouseWheelEvent` como argumento.

La clase `MouseWheelEvent` (una subclase de `MouseEvent`) contiene métodos que permiten al manejador de eventos obtener información acerca de la cantidad de rotación de la rueda.

APP

```
import javax.swing.JFrame;
public class App
{
    public static void main(String[] args)
    {
        MarcoRastreadorRaton marcoRastreadorRaton = new
        MarcoRastreadorRaton();
        marcoRastreadorRaton.setDefaultCloseOperation(JFrame.EXIT_0
        N_CLOSE);
        marcoRastreadorRaton.setSize(300, 100);
        marcoRastreadorRaton.setVisible(true);
    }
} // Fin de App Prueba Label
```

MARCO RASTREADOR RATON

```
//Manejo de eventos de ratón.
import java.awt.Color;
import java.awt.BorderLayout;
import java.awt.event.MouseListener;
import java.awt.event.MouseMotionListener;
import java.awt.event.MouseEvent;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
public class MarcoRastreadorRaton extends JFrame {
    private final JPanel panelRaton; // panel en el que ocurrirán los eventos
    de ratón
    private final JLabel barraEstado; // muestra información de los eventos
    //El constructor de MarcoRastreadorRaton establece la GUI y
    //registra los manejadores de eventos de ratón
    public MarcoRastreadorRaton()
    {
        super("Demostracion de los eventos de raton");
        panelRaton = new JPanel();
        panelRaton.setBackground(Color.WHITE);
        add(panelRaton, BorderLayout.CENTER); // agrega el panel a JFrame
        barraEstado = new JLabel("Raton fuera de JPanel");
        add(barraEstado, BorderLayout.SOUTH); // agrega etiqueta a JFrame
        //crea y registra un componente de escucha para los eventos de ratón y de
        su
        //movimiento
        ManejadorRaton manejador = new ManejadorRaton();
        panelRaton.addMouseListener(manejador);
        panelRaton.addMouseMotionListener(manejador);
    }
    private class ManejadorRaton implements MouseListener, MouseMotionListener
    {
        //Los manejadores de eventos de MouseListener
        //manejan el evento cuando se suelta el ratón justo después de oprimir el
        botón
        @Override
        public void mouseClicked(MouseEvent evento)
        {
            barraEstado.setText(String.format("Se hizo clic en [%d, %d]",
            evento.getX(),
            evento.getY()));
        }
        //maneja evento cuando se oprime el ratón
        @Override
        public void mousePressed(MouseEvent evento)
        {
            barraEstado.setText(String.format("Se oprimio en [%d, %d]", evento.getX(),
            evento.getY()));
        }
        //maneja evento cuando se suelta el botón del ratón
```

```
@Override
    public void mouseReleased(MouseEvent evento)
    {
        barraEstado.setText(String.format("Se solto en [%d, %d]", evento.getX(),
        evento.getY()));
    }
    // maneja evento cuando el ratón entra al área
    @Override
    public void mouseEntered(MouseEvent evento)
    {
        barraEstado.setText(String.format("Raton entro en [%d, %d]", evento.getX(),
        evento.getY()));
        panelRaton.setBackground(Color.GREEN);
    }
    // maneja evento cuando el ratón sale del área
    @Override
    public void mouseExited(MouseEvent evento)
    {
        barraEstado.setText("Raton fuera de JPanel");
        panelRaton.setBackground(Color.WHITE);
    }
    // Los manejadores de eventos de MouseMotionListener manejan
    // el evento cuando el usuario arrastra el ratón con el botón oprimido
    @Override
    public void mouseDragged(MouseEvent evento)
    {
        barraEstado.setText(String.format("Se arrastro en [%d, %d]", evento.getX(),
        evento.getY()));
    }
    // maneja evento cuando el usuario mueve el ratón
    @Override
    public void mouseMoved(MouseEvent evento)
    {
        barraEstado.setText(String.format("Se movio en [%d, %d]", evento.getX(),
        evento.getY()));
    }
    // fin de la clase interna ManejadorRaton
    // fin de la clase MarcoRastreadorRaton
```

CLASES ADAPTADORAS

Muchas de las interfaces de escucha de eventos, como `MouseListener` y `MouseMotionListener`, contienen varios métodos. No siempre es deseable declarar todos los métodos en una interfaz de escucha de eventos.

Por ejemplo, una aplicación podría necesitar solamente el manejador `mouseClicked` de la interfaz `MouseListener`.

Una clase adaptadora implementa a una interfaz y proporciona una implementación predeterminada (con un cuerpo vacío para los métodos) de todos los métodos en la interfaz.

APP

```
import javax.swing.JFrame;
public class App
{
    public static void main(String[] args) {
        MarcoDetallesRaton marcoDetallesRaton = new
        MarcoDetallesRaton();
        marcoDetallesRaton.setDefaultCloseOperation(JFrame.EXIT_ON_
        CLOSE);
        marcoDetallesRaton.setSize(400, 150);
        marcoDetallesRaton.setVisible(true);
    }
} // fin de la clase DetallesRaton
```

MARCO DETALLES

RATON

```
import java.awt.BorderLayout;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;
import javax.swing.JFrame;
import javax.swing.JLabel;
public class MarcoDetallesRaton extends JFrame
{
    private String detalles; // String que se
    muestra en barraEstado
    private final JLabel barraEstado; // JLabel que
    aparece en la parte inferior de la
    //ventana
    //constructor establece String de la barra de
    título y registra componente de
    //escucha del ratón
    public MarcoDetallesRaton()
    {
        super("Clics y botones del raton");
        barraEstado = new JLabel("Haga clic en el
        raton");
        add(barraEstado, BorderLayout.SOUTH);
        addMouseListener(new ManejadorClicRaton()); //
        agrega el manejador
    }
    //clase interna para manejar los eventos del
    ratón
    private class ManejadorClicRaton extends
    MouseAdapter
    {
        //maneja evento de clic del ratón y determina
        cuál botón se oprimió
```

```
@Override
public void mouseClicked(MouseEvent evento)
{
    int xPos = evento.getX(); // obtiene posición x
    del ratón en caso requerirse
    int yPos = evento.getY(); // obtiene posición y
    del ratón en caso requerirse
    detalles = String.format("Se hizo clic %d
    vez(vezes)",
    evento.getClickCount());
    if (evento.getButton() == MouseEvent.BUTTON1) {
        detalles += " con el boton izquierdo del raton";
    }
    if (evento.getButton() == MouseEvent.BUTTON2) {
        detalles += " con el boton central del raton";
    }
    if (evento.getButton() == MouseEvent.BUTTON3) {
        detalles += " con el boton derecho del raton";
    }
    barraEstado.setText(detalles); // muestra
    mensaje en barraEstado
} // fin de la clase MarcoDetallesRaton
}
```

MANEJO DE EVENTOS DE TECLAS

La interfaz `KeyListener` maneja eventos de teclas.

Estos eventos se generan cuando se oprimen y sueltan las teclas en el teclado.

Una clase que implementa a `KeyListener` debe proporcionar declaraciones para los métodos `keyPressed`, `keyReleased` y `keyTyped`, cada uno de los cuales recibe un objeto `KeyEvent` como argumento.

- El método `keyPressed` es llamado en respuesta a la acción de oprimir cualquier tecla.
- El método `keyTyped` es llamado en respuesta a la acción de oprimir una tecla que no sea una tecla de acción. (Las teclas de acción son cualquier tecla de dirección, Inicio, Fin, RePág, AvPág, cualquier tecla de función, etc.).
- El método `keyReleased` es llamado cuando la tecla se suelta después de un evento `keyPressed` o `keyTyped`.

APP

```
import javax.swing.JFrame;
public class App
{
    public static void main(String[] args) {
        MarcoDemoTeclas marcoDemoTeclas = new MarcoDemoTeclas();
        marcoDemoTeclas.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        marcoDemoTeclas.setSize(350, 100);
        marcoDemoTeclas.setVisible(true);
    }
} // fin de la clase DemoTeclas
```

MARCO DEMO TECLAS

```
import java.awt.Color;
import java.awt.event.KeyListener;
import java.awt.event.KeyEvent;
import javax.swing.JFrame;
import javax.swing.JTextArea;
public class MarcoDemoTeclas extends JFrame implements
KeyListener {
    private String linea1 = ""; // primera línea del área de
    texto
    private String linea2 = ""; // segunda línea del área de
    texto
    private String linea3 = ""; // tercera línea del área de
    texto
    private JTextArea areaTexto; // área de texto para mostrar
    la salida
    // constructor de MarcoDemoTeclas
    public MarcoDemoTeclas()
    {
        super("Demostracion de los eventos de pulsacion de
        teclas");
        areaTexto = new JTextArea(10, 15); // establece el objeto
        JTextArea
        areaTexto.setText("Oprima cualquier tecla en el
        teclado...");
        areaTexto.setEnabled(false);
        areaTexto.setDisabledTextColor(Color.BLACK);
        add(areaTexto); // agrega el área de texto a JFrame
        addKeyListener(this); // permite al marco procesar los
        eventos de teclas
    }
    // maneja el evento de oprimir cualquier tecla
    @Override
    public void keyPressed(KeyEvent evento)
    {
        linea1 = String.format("Tecla oprimida: %s",
        KeyEvent.getKeyText(evento.getKeyCode())); // muestra la
        tecla oprimida
        establecerLineas2y3(evento); // establece las líneas de
        salida dos y tres
    }
    // maneja el evento de liberar cualquier tecla
    @Override
    public void keyReleased(KeyEvent evento)
    {
        linea1 = String.format("Tecla liberada: %s",
        KeyEvent.getKeyText(evento.getKeyCode())); // muestra la
        tecla liberada
        establecerLineas2y3(evento); // establece las líneas de
        salida dos y tres
    }
    // maneja el evento de oprimir una tecla de acción
    @Override
    public void keyTyped(KeyEvent evento)
    {
        linea1 = String.format("Tecla oprimida: %s",
        evento.getKeyChar());
        establecerLineas2y3(evento); // establece las líneas de
        salida dos y tres
    }
    // establece las líneas de salida dos y tres
    private void establecerLineas2y3(KeyEvent evento)
    {
        linea2 = String.format("Esta tecla %s es una tecla de
        accion",
        (evento.isActionKey() ? "" : "no "));
        String temp =
        KeyEvent.getModifiersExText(evento.getModifiersEx());
        linea3 = String.format("Teclas modificadoras oprimidas:
        %s",
        (temp.equals("") ? "ninguna" : temp)); // imprime
        modificadoras
        areaTexto.setText(String.format("%s\n%s\n%s\n", linea1,
        linea2, linea3));
        // imprime tres líneas de texto
    }
    // fin de la clase MarcoDemoTeclas
}
```